

# DATABASE MANAGEMENT SYSTEMS : NOTES W11

## What is Backup and Recovery?

- A **Backup** of a database is a representative copy of data containing all necessary contents of a database such as data files and control files
  - Unexpected database failures, especially those due to factors beyond our control, are unavoidable. Hence, it is important to keep a backup of the entire database
  - There are two major types of backups:
    - **Physical Backup:** A copy of physical database files such as data, control files, log files, and archived redo logs.
    - **Logical Backup:** A copy of logical data that is extracted from a database consisting of tables, procedures, views, functions, etc.
- **Recovery** is the process of restoring the database to its latest known consistent state after a system failure occurs.
  - A Database Log records all transactions in a sequence. Recovery using logs is quite popular in databases
  - A typical log file contains information about transactions to execute, transaction states, and modified values

## Why is backup necessary?

- **Disaster Recovery**
  - Data loss can occur due to various reasons like hardware failures, malware attacks, environmental & physical factors or a simple human error
- **Client-Side Changes**
  - Clients may want to modify the existing application to serve their business's dynamic needs
  - Developers might need to restore a previous version of the database in order to such address such requirements
- **Auditing**
  - From an auditing perspective, you need to know what your data or schema looked like at some point in the past
  - For instance, if your organization happens to get involved in a lawsuit, it may want to have a look at an earlier snapshot of the database.
- **Downtime**
  - Without backup, system failures lead to data loss, which in turn results in application downtime
  - This leads to bad business reputation

## Backup Data: Types of Backup Data

- **Business Data** includes personal information of clients, employees, contractors etc. along with details about places, things, events and rules related to the business.
- **System Data** includes specific environment/configuration of the system used for specialised development purposes, log files, software dependency data, disk images.
- **Media** files like photographs, videos, sounds, graphics etc. need backing up. Media files are typically much larger in size.

# Backup Strategies

## Types of Backup Strategies: Full Backup

- **Full Backup** backs up everything. This is a complete copy, which stores all the objects of the database: tables, procedures, functions, views, indexes etc. Full backup can restore all components of the database system as it was at the time of crash.
- **A full backup must be done at least once before any of the other type of backup**
- The frequency of a full backup depends on the type of application. For instance, a full backup is done on a **daily basis** for applications in which one or more of the following is/are true:
  - Either 24/7 availability is not a requirement, or system availability is not affected as a consequence of backups.
  - A complete backup takes a minimum amount of media, i.e. the backup data is not too large.
  - Backup/system administrators may not be available on a daily basis, and therefore a primary goal is to reduce to a bare minimum the amount of media required to complete a restore.
- **Full Backup: Advantages**
  - Recovery from a full backup involves a consolidated read from a single backup
  - Generally, there will not be any dependency between two consecutive backups.
  - Effectively, the loss of a single day's backup does not affect the ability to recover other backups
  - It is relatively easy to setup, configure and maintain
- **Full Backup: Disadvantages**
  - The backup takes largest amount of time among all types of backups
  - This results in longest system downtime during the backup process
  - It uses largest amount of storage media per backup

## Types of Backup Strategies: Incremental Backup

- **Incremental** backup targets only those files or items that have changed since the last backup. This often results in smaller backups and needs shorter duration to complete the backup process.
- For instance, a 2 TB database may only have a 5% change during the day. With incremental database backups, the amount backed up is typically only a little more than the actual amount of **changed data** in the database.
- For most organizations, **a full backup is done once a week, and incremental backups are done for the rest of the time**. This might mean a backup schedule as shown below

| <i>Friday</i> | <i>Saturday</i> | <i>Sunday</i> | <i>Monday</i> | <i>Tuesday</i> | <i>Wednesday</i> | <i>Thursday</i> |
|---------------|-----------------|---------------|---------------|----------------|------------------|-----------------|
| Full          | Incremental     | Incremental   | Incremental   | Incremental    | Incremental      | Incremental     |

- **This ensures a minimum backup window during peak activity times, with a longer backup window during non-peak activity times.**
- **Incremental Backup: Advantages**
  - Less storage is used per backup
  - The downtime due to backup is minimized
  - It provides considerable cost reductions over full backups
- **Incremental Backup: Disadvantages**
  - It requires more effort and time during recovery

- A complete system recovery needs a full backup to start with
- It cannot be done without the full backups and all incremental backups in between
- If any of the intermediate incremental backups are lost, then the recovery cannot be 100%

## Types of Backup Strategies: Differential Backup

- **Differential** backup backs up all the changes that have occurred since the most recent full backup regardless of what backups have occurred in between
- This “rolls up” multiple changes into a single backup job which sets the basis for the next incremental backup
  - As a differential backup does not back up everything, this backup process usually runs quicker than a full backup
  - The longer the age of a differential backup, the larger the size of its backup window
- To evaluate how differential backups might work within an environment, consider the sample backup schedule shown in the figure below.

|        |             |             |             |              |             |             |
|--------|-------------|-------------|-------------|--------------|-------------|-------------|
| Friday | Saturday    | Sunday      | Monday      | Tuesday      | Wednesday   | Thursday    |
| Full   | Incremental | Incremental | Incremental | Differential | Incremental | Incremental |

a) The incremental backup on Saturday backs up all files that have changed since the full backup on Friday. Likewise all changes since Saturday and Sunday is backed up on Sunday and Monday’s incremental backup respectively.

b) On Tuesday, a differential backup is performed. This backs up all files that have changed since the full backup on Friday. A recovery on Wednesday should only require data from the full and differential backups, **skipping the Saturday/Sunday/Monday incremental backups**.

**Recovery on any given day only needs the data from the full backup and the most recent differential backup**



- **Differential Backup: Advantages**
  - Recoveries require fewer backup sets.
  - Provide better recovery options when full backups are run rarely (for example, only monthly)
- **Differential Backup: Disadvantages**
  - Although the number of backup sets required for recovery is less but in differential backups the amount of storage media required may exceed the storage media required for incremental backups
  - If done after quite a long time, differential backups can even reach the size of a full backup

## Types of Backup Strategies: Illustrative Example

- The figure below depicts which of the updated files of the database will be backed up in each respective type of backup throughout a span of 5 days as indicated.

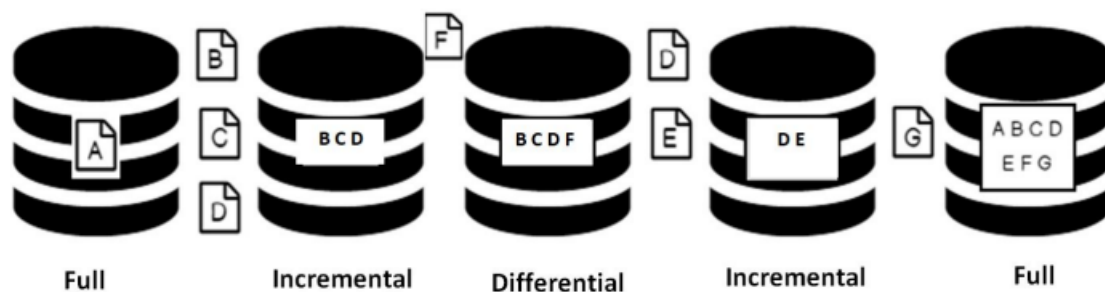


Figure: Backup Types

## Case: Monthly Data Backup Schedule

Consider the following backup schedule for a month:

| Sunday  | Monday  | Tuesday | Wednesday | Thursday | Friday  | Saturday |
|---------|---------|---------|-----------|----------|---------|----------|
| 1/Full  | 2/Incr  | 3/Incr  | 4/Incr    | 5/Incr   | 6/Incr  | 7/Incr   |
| 8/Diff  | 9/Incr  | 10/Incr | 11/Incr   | 12/Incr  | 13/Incr | 14/Incr  |
| 15/Diff | 16/Incr | 17/Incr | 18/Incr   | 19/Incr  | 20/Incr | 21/Incr  |
| 22/Diff | 23/Incr | 24/Incr | 25/Incr   | 26/Incr  | 27/Incr | 28/Incr  |
| 29/Diff | 30/Incr | 31/Incr |           |          |         |          |

### • Inference

- Here full backups are performed once per month, but with differentials being performed weekly, the maximum number of backups required for a **complete system recovery** at any point will be **one full backup, one differential backup, and six incremental backups**
- A full system recovery will never need more than the full backup from the start of the month, the differential backup at the start of the relevant week, and the incremental backups performed during the week
- If a policy were used whereby **full backups were done on the first of the month, and incrementals for the rest of the month**, a complete system recovery on last day of month will need as many as **31 backup sets**
- Thus differential backups can improve efficiency of recovery when planned properly

## Hot Backup

- Till now we have learnt about backup strategies which cannot happen simultaneously with a running application
- In systems where high availability is a requirement **Hot backup** is preferable **wherever possible**
- **Hot backup** refers to keeping a database up and running while the backup is performed concurrently
  - Such a system usually has a module or plug-in that allows the database to be backed up while staying available to end users
  - Databases which stores transactions of **asset management companies, hedge funds, high frequency trading companies etc.** try to implement Hot backups as these data are highly dynamic and the operations run 24x7
  - **Real time systems like sensor and actuator data in embedded devices, satellite transmissions etc.** also use Hot backup
- **Hot Backup: Advantages**
  - The database is always available to the end user.
  - Point-in-time recovery is easier to achieve in Hot backup systems.
  - Most efficient while dealing with dynamic and modularized data.

- **Hot Backup: Disadvantages**

- May not be feasible when the data set is huge and monolithic.
- Fault tolerance is less. Occurrence of any error on the fly can terminate the whole backup process.
- Maintenance and setup cost is high

## Transactional Logging as Hot Backup

- In regular database systems, hot backup is mainly used for Transaction Log Backup.
- **Cold backup** strategies like **Differential, Incremental** are preferred for Data backup. The reason is evident from the disadvantages of Hot backup.
- **Transactional Logging** is used in circumstances where a possibly inconsistent backup is taken, but another file generated and backed up (after the database file has been fully backed up) can be used to restore consistency.
- The information regarding **data backup versions** while recovery at a **given point** can be inferred from the Transactional Log backup set.
- Thus they play a vital role in **database recovery**.

## Failure Classification

### Database System Recovery

- All database reads/writes are within a transaction
- Transactions have the “ACID” properties
  - Atomicity - all or nothing
  - Consistency - preserves database integrity
  - Isolation - execute as if they were run alone
  - Durability - results are not lost by a failure
- Concurrency Control guarantees I, contributes to C
- Application program guarantees C
- Recovery subsystem guarantees A & D, contributes to C

## Failure Classification

- **Transaction failure:**
  - **Logical errors:** transaction cannot complete due to some internal error condition
  - **System errors:** the database system must terminate an active transaction due to an error condition (for example, deadlock)
- **System crash:** a power failure or other hardware or software failure causes the system to crash
  - **Fail-stop assumption:** non-volatile storage contents are assumed to not be corrupted as result of a system crash
    - Database systems have numerous integrity checks to prevent corruption of disk data
- **Disk failure:** a head crash or similar disk failure destroys all or part of disk storage
  - Destruction is assumed to be detectable
    - Disk drives use checksums to detect failures

## Recovery Algorithms

- Consider transaction  $T_i$  that transfers \$50 from account A to account B

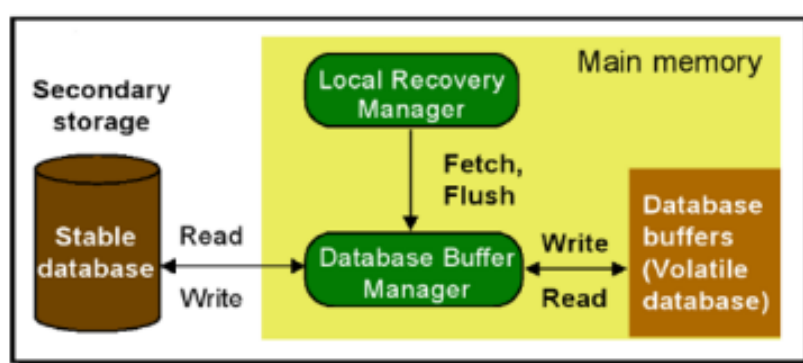
- Two updates: subtract 50 from A and add 50 to B
- Transaction  $T_i$  requires updates to A and B to be output to the database
  - A failure may occur after one of these modifications have been made but before both of them are made
  - Modifying the database without ensuring that the transaction will commit may leave the database in an inconsistent state
  - Not modifying the database may result in lost updates if failure occurs just after transaction commits
- Recovery algorithms have two parts
  - a) Actions taken during normal transaction processing to ensure enough information exists to recover from failures
  - b) Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability

## Storage Structure

- **Volatile Storage:**
  - does not survive system crashes
  - examples: main memory, cache memory
- **Non-volatile Storage:**
  - survives system crashes
  - examples: disk, tape, flash memory, non-volatile (battery backed up) RAM
  - but may still fail, losing data
- **Stable Storage:**
  - a mythical form of storage that survives all failures
  - approximated by maintaining multiple copies on distinct non-volatile media

## Stable Storage Implementation

- Maintain multiple copies of each block on separate disks
  - copies can be at remote sites to protect against disasters such as fire or flooding
- Failure during data transfer can still result in inconsistent copies. Block transfer can result in
  - Successful completion
  - Partial failure: destination block has incorrect information
  - Total failure: destination block was never updated
- Protecting storage media from failure during data transfer (one solution):
  - Execute output operation as follows (assuming two copies of each block):
    - Write the information onto the 1st physical block
    - When the 1st write is successful, write the same information onto the 2nd physical block
    - The output is completed only after the second write successfully completes

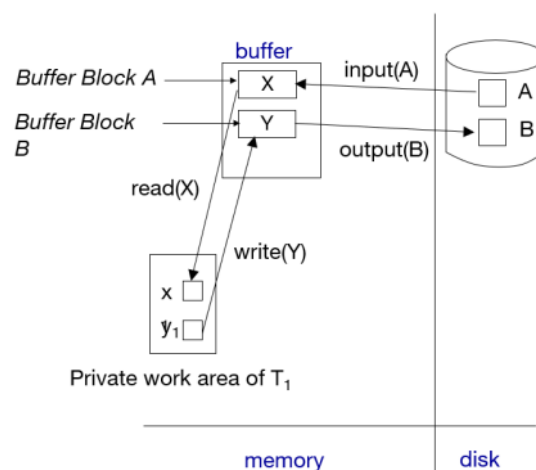


## Protecting storage media from failure during data transfer (cont.):

- Copies of a block may differ due to failure during output operation
- To recover from failure:
  - First find inconsistent blocks:
    - Expensive solution : Compare the two copies of every disk block
    - Better solution:
      - ✓ Record in-progress disk writes on non-volatile storage (Non-volatile RAM or special area of disk)
      - ✓ Use this information during recovery to find blocks that may be inconsistent, and only compare copies of these
      - ✓ Used in hardware RAID systems
  - If either copy of an inconsistent block is detected to have an error (bad checksum), overwrite it by the other copy
  - If both have no error, but are different, overwrite the second block by the first block

## Data Access

- **Physical Blocks** are those blocks residing on the disk
- **System Buffer Blocks** are the blocks residing temporarily in main memory
- Block movements between disk and main memory are initiated through the following two operations:
  - **input(B)** transfers the physical block B to main memory
  - **output(B)** transfers the buffer block B to the disk, and replaces the appropriate physical block there
- We assume, for simplicity, that each data item fits in, and is stored inside, a single block
- Each transaction  $T_i$  has its private work-area in which local copies of all data items accessed and updated by it are kept
  - $T_i$ 's local copy of a data item X is denoted by  $x_i$
  - BX denotes block containing X
- Transferring data items between system buffer blocks and its private work-area done by:
  - **read(X)** assigns the value of data item X to the local variable  $x_i$
  - **write(X)** assigns the value of local variable  $x_i$  to data item X in the buffer block
- Transactions
  - Must perform **read(X)** before accessing X for the first time (subsequent reads can be from local copy)
  - The **write(X)** can be executed at any time before the transaction commits
- Note that **output(B<sub>x</sub>)** need not immediately follow **write(X)**. System can perform the output operation when it deems fit

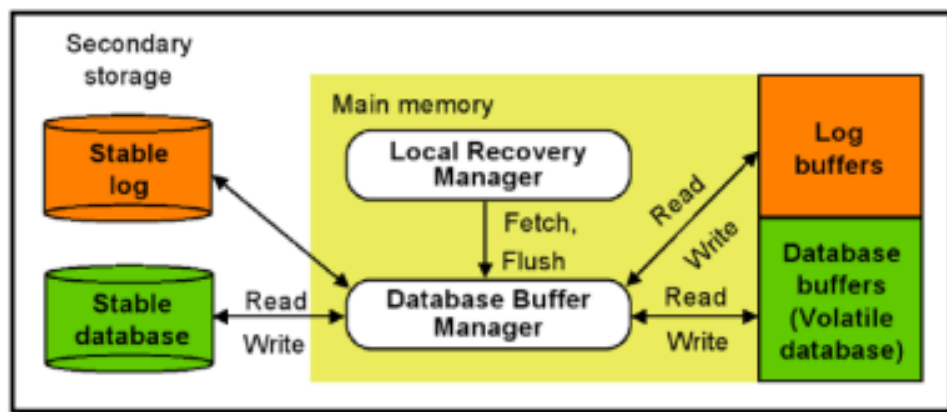


## Recovery and Atomicity

- To ensure atomicity despite failures, we first output information describing the modifications to stable storage without modifying the database itself
- We study **Log-based Recovery Mechanisms**
  - We first present key concepts
  - And then present the actual recovery algorithm
- Less used alternative: **Shadow Paging**
- In this Module we assume serial execution of transactions
- In the next Module, we consider the case of concurrent transaction execution

## Log-Based Recovery

- A **log** is kept on stable storage
  - The log is a sequence of **log records**, which maintains information about update activities on the database
- When transaction  $T_i$  starts, it registers itself by writing a record  $\langle T_i \text{ start} \rangle$  to the log
- Before  $T_i$  executes **write(X)**, a log record  $\langle T_i, X, V1, V2 \rangle$  is written, where  $V1$  is the value of  $X$  before the write (**old value**), and  $V2$  is the value to be written to  $X$  (**new value**)
- When  $T_i$  finishes its last statement, the log record  $\langle T_i \text{ commit} \rangle$  is written



## Database Modification Schemes

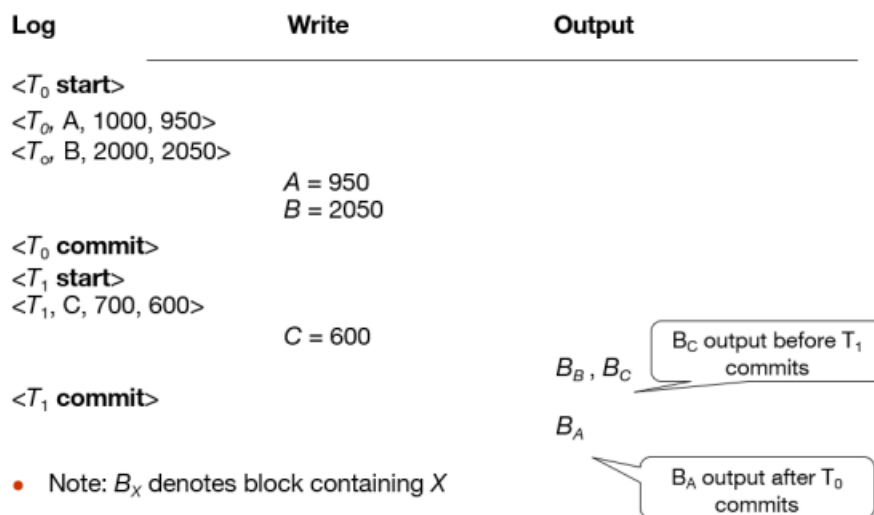
- The **immediate-modification** scheme allows updates of an uncommitted transaction to be made to the buffer, or the disk itself, before the transaction commits
  - Update log record must be written before a database item is written
    - We assume that the log record is output directly to stable storage
  - Output of updated blocks to disk storage can take place at any time before or after transaction commit
  - Order in which blocks are output can be different from the order in which they are written
- The **deferred-modification** scheme performs updates to buffer/disk only at the time of transaction commit
  - Simplifies some aspects of recovery
  - But has overhead of storing local copy
- We cover here only the immediate-modification scheme

## Transaction Commit

- A transaction is said to have committed when its commit log record is output to stable storage. All previous log records of the transaction must have been output already

- Writes performed by a transaction may still be in the buffer when the transaction commits, and may be output later

## Immediate Database Modification Example



## Undo and Redo Operations

- Undo** of a log record  $\langle T_i, X, V1, V2 \rangle$  writes the **old** value  $V1$  to  $X$
- Redo** of a log record  $\langle T_i, X, V1, V2 \rangle$  writes the **new** value  $V2$  to  $X$
- Undo and Redo of Transactions**
  - undo( $T_i$ )** restores the value of all data items updated by  $T_i$  to their old values, going backwards from the last log record for  $T_i$ 
    - Each time a data item  $X$  is restored to its old value  $V$  a special log record (called **redo-only**)  $\langle T_i, X, V \rangle$  is written out
    - When undo of a transaction is complete, a log record  $\langle T_i \text{ abort} \rangle$  is written out (to indicate that the undo was completed)
  - redo( $T_i$ )** sets the value of all data items updated by  $T_i$  to the new values, going forward from the first log record for  $T_i$ 
    - No logging is done in this case
- The **undo** and **redo** operations are used in several different circumstances:
  - The undo is used for transaction rollback during normal operation
    - in case a transaction cannot complete its execution due to some logical error
  - The **undo** and **redo** operations are used during recovery from failure
- We need to deal with the case where during recovery from failure another failure occurs prior to the system having fully recovered

## Undo and Redo on Normal Transaction Rollback

- Let  $T_i$  be the transaction to be rolled back
- Scan log backwards from the end, and for each log record of  $T_i$  of the form  $\langle T_i, X_j, V1, V2 \rangle$ 
  - Perform the undo by writing  $V1$  to  $X_j$ ,
  - Write a log record  $\langle T_i, X_j, V1 \rangle$ 
    - such log records are called **Compensation Log Records**
- Once the record  $\langle T_i \text{ start} \rangle$  is found stop the scan and write the log record  $\langle T_i \text{ abort} \rangle$

## Undo and Redo on Recovering from Failure

- When recovering after failure:
  - Transaction  $T_i$  needs to be undone if the log
    - contains the record  $\langle T_i \text{ start} \rangle$ ,
    - but does not contain either the record  $\langle T_i \text{ commit} \rangle$  or  $\langle T_i \text{ abort} \rangle$
  - Transaction  $T_i$  needs to be redone if the log
    - contains the records  $\langle T_i \text{ start} \rangle$
    - and contains the record  $\langle T_i \text{ commit} \rangle$  or  $\langle T_i \text{ abort} \rangle$
  - It may seem strange to redo transaction  $T_i$  if the record  $\langle T_i \text{ abort} \rangle$  record is in the log
    - To see why this works, note that if  $\langle T_i \text{ abort} \rangle$  is in the log, so are the redo-only records written by the undo operation. Thus, the end result will be to undo  $T_i$ 's modifications in this case. This slight redundancy simplifies the recovery algorithm and enables faster overall recovery time
    - such a redo redoes all the original actions including the steps that restored old value – Known as **Repeating History**

## Immediate Modification Recovery Example

Below we show the log as it appears at three instances of time.

|                                      |                                      |                                      |
|--------------------------------------|--------------------------------------|--------------------------------------|
| $\langle T_0 \text{ start} \rangle$  | $\langle T_0 \text{ start} \rangle$  | $\langle T_0 \text{ start} \rangle$  |
| $\langle T_0, A, 1000, 950 \rangle$  | $\langle T_0, A, 1000, 950 \rangle$  | $\langle T_0, A, 1000, 950 \rangle$  |
| $\langle T_0, B, 2000, 2050 \rangle$ | $\langle T_0, B, 2000, 2050 \rangle$ | $\langle T_0, B, 2000, 2050 \rangle$ |
|                                      | $\langle T_0 \text{ commit} \rangle$ | $\langle T_0 \text{ commit} \rangle$ |
|                                      | $\langle T_1 \text{ start} \rangle$  | $\langle T_1 \text{ start} \rangle$  |
|                                      | $\langle T_1, C, 700, 600 \rangle$   | $\langle T_1, C, 700, 600 \rangle$   |
|                                      |                                      | $\langle T_1 \text{ commit} \rangle$ |
| (a)                                  | (b)                                  | (c)                                  |

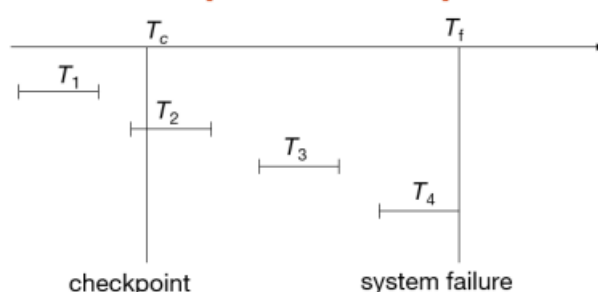
Recovery actions in each case above are:

- undo ( $T_0$ ): B is restored to 2000 and A to 1000, and log records  $\langle T_0, B, 2000 \rangle$ ,  $\langle T_0, A, 1000 \rangle$ ,  $\langle T_0, \text{abort} \rangle$  are written out
- redo ( $T_0$ ) and undo ( $T_1$ ): A and B are set to 950 and 2050 and C is restored to 700. Log records  $\langle T_1, C, 700 \rangle$ ,  $\langle T_1, \text{abort} \rangle$  are written out
- redo ( $T_0$ ) and redo ( $T_1$ ): A and B are set to 950 and 2050 respectively. Then C is set to 600.

## Checkpoints

- Redoing/undoing all transactions recorded in the log can be very slow
  - Processing the entire log is time-consuming if the system has run for a long time
  - We might unnecessarily redo transactions which have already output their updates to the database
- Streamline recovery procedure by periodically performing **checkpointing**
- All updates are stopped while doing checkpointing
  - Output all log records currently residing in main memory onto stable storage
  - Output all modified buffer blocks to the disk
  - Write a log record  $\langle \text{checkpoint } L \rangle$  onto stable storage where L is a list of all transactions active at the time of checkpoint
- During recovery we need to consider only the most recent transaction  $T_i$  that started before the checkpoint, and transactions that started after  $T_i$ 
  - Scan backwards from end of log to find the most recent  $\langle \text{checkpoint } L \rangle$

- Only transactions that are in L or started after the checkpoint need to be redone or undone
- Transactions that committed or aborted before the checkpoint already have all their updates output to stable storage
- Some earlier part of the log may be needed for undo operations
  - Continue scanning backwards till a record  $\langle T_i \text{ start} \rangle$  is found for every transaction  $T_i$  in L
  - Parts of log prior to earliest  $\langle T_i \text{ start} \rangle$  record above are not needed for recovery, and can be erased whenever desired



- Any transactions that committed before the last checkpoint should be ignored
  - T1 can be ignored (updates already output to disk due to checkpoint)
- Any transactions that committed since the last checkpoint need to be redone
  - T2 and T3 redone
- Any transaction that was running at the time of failure needs to be undone and restarted
  - T4 undone

## Transactional Logging

### Hot Backup: Recap

- In systems where high availability is a requirement **Hot backup** is preferable **wherever possible**
- **Hot backup** refers to keeping a database up and running while the backup is performed concurrently
  - Such a system usually has a module or plug-in that allows the database to be backed up while staying available to end users
  - Databases which stores transactions of **asset management companies, hedge funds, high frequency trading companies** etc. try to implement Hot backups as these data are highly dynamic and the operations run 24x7
  - Real time systems **like sensor and actuator data in embedded devices, satellite transmissions** etc. also use Hot backup

### Transactional Logging as Hot Backup

- In regular database systems, **Hot Backup** is mainly used for Transaction Log Backup
- **Cold backup** strategies like **Differential, Incremental** are preferred for **Data backup** The reason is evident from the disadvantages of Hot backup
- **Transactional Logging** is used in circumstances where a possibly inconsistent backup is taken, but another file generated and backed up (after the database file has been fully backed up) can be used to restore consistency
- The information regarding **data backup versions** while recovery at a **given point** can be inferred from the Transactional Log backup set
- Thus they play a vital role in **database recovery**

## Transactional Logging with Recovery: Example

- To understand how **Transactional Logging** works we consider Figure 1 that represents a chunk of a database just before a backup has been started
- While the backup is in progress, modifications may continue to occur to the database. For example, a request to modify the data at location **"4325"** to '0' arrives.
- When a request comes through to modify a part of the DB, the modifications will be written in the **given order compulsorily** 1 Transaction Log 2 Database (itself) This is depicted in Figure 2
- If a crash occurs **before writing to the database** then the inconsistent backed up file is recovered first, and then the pending modifications in the transaction log (backed up\*) are applied to re-establish consistency

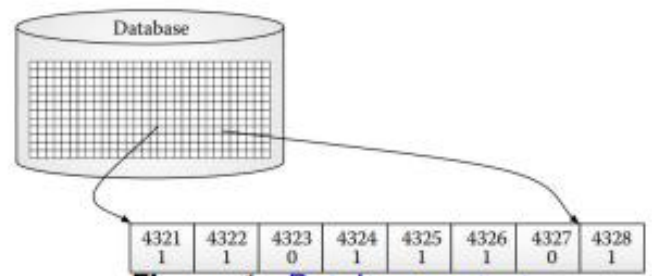


Figure 1: Database content

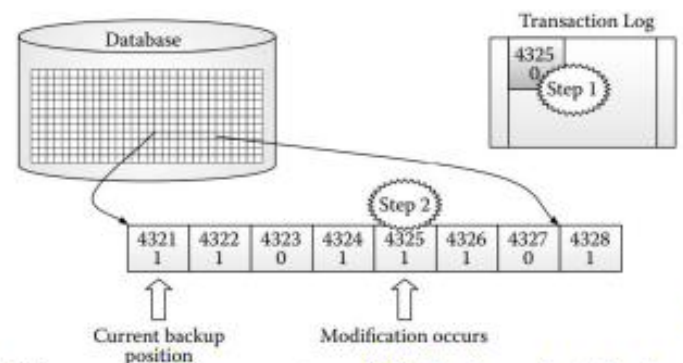


Figure 2: Changes to a DB during a hot backup

**\*Note: The Transactional Log itself is backed up using Hot Backup the Data is backed up incrementally**

Consider in the previous scenario before the occurrence of crash, another request modifies the content of location **"4321"** to '0'. Incidentally, **this change gets written in the database itself** (recall: Immediate Modification). This is indicated in Figure 3

- Figure 3 is the state of the database after which the system crashes. Note that this part has already been backed up, and hence, the backup is inconsistent with the database.

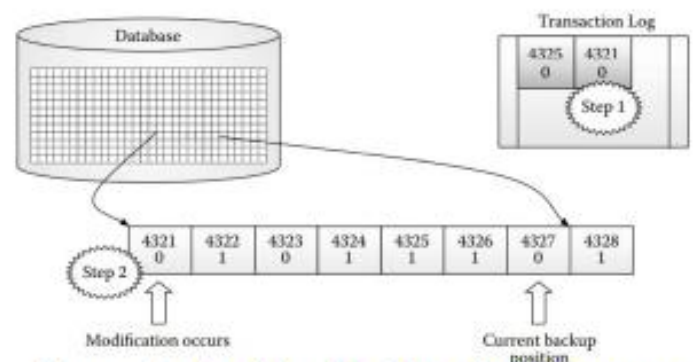


Figure 3: Applying Tr. logs during recovery

- Recovery Phase:
  - Data recovery is done from the last data backup set (Figure 1)
  - Log recovery is done from the Transaction Log backup set. It will be same as the current transaction log because of Hot backup
  - Figure 4 shows the recovered database and log
- The recovered database is inconsistent. To re-establish consistency all transaction logs generated between the start of the backup and the end of the backup must be **replayed**

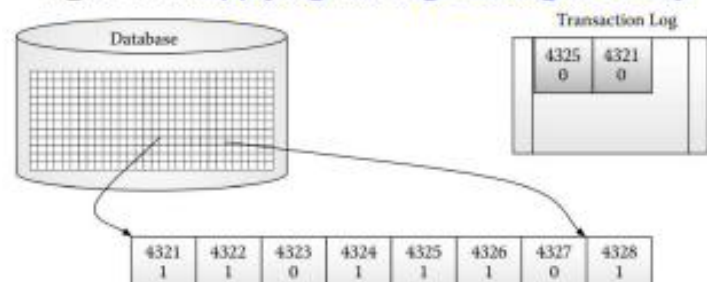
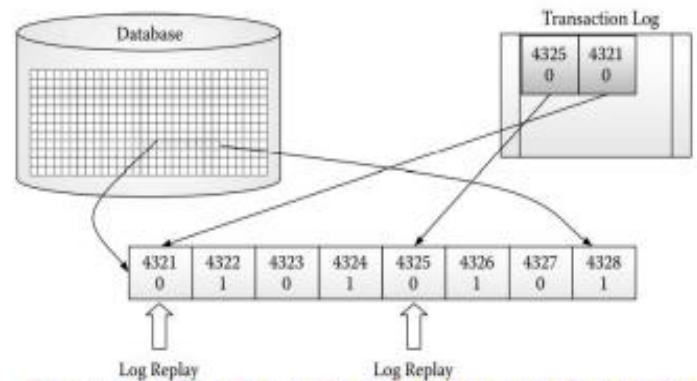


Figure 4: Recovered DB files and Tr. logs

- When using transactional logging we distinguish between recover and restore:
  - **Recover:** retrieve from the backup media the database files and transaction logs, and
  - **Restore:** reapply database consistency based on the transaction logs
- For our restore process, we recover inconsistent database files and completed transaction logs. The recovered files will resemble the configuration shown in Figure 4
- The final database state after replaying log on the recovered database is displayed in Figure 5
- **The state of database is consistent**
- Note that an unnecessary log replay is shown occurring for block 4325. Whether such replays will occur is dependent on the database being used. For instance, a database vendor might choose to replay all logs because it would be faster than first determining whether a particular logged activity needs to be replayed
- Once all transaction logs have been replayed, the database is said to have been restored, that is, it is at a point where it can now be opened for user access



**Figure: 5:** Database restore process via log replay

## Recovery Algorithm

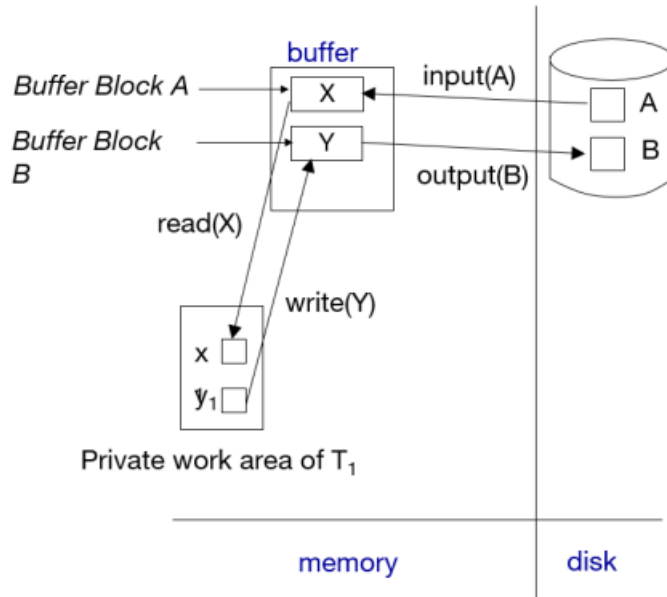
### Recovery Schemes

- **So far:**
  - We covered key concepts
  - We assumed serial execution of transactions
- **Now:**
  - We discuss concurrency control issues
  - We present the components of the basic recovery algorithm

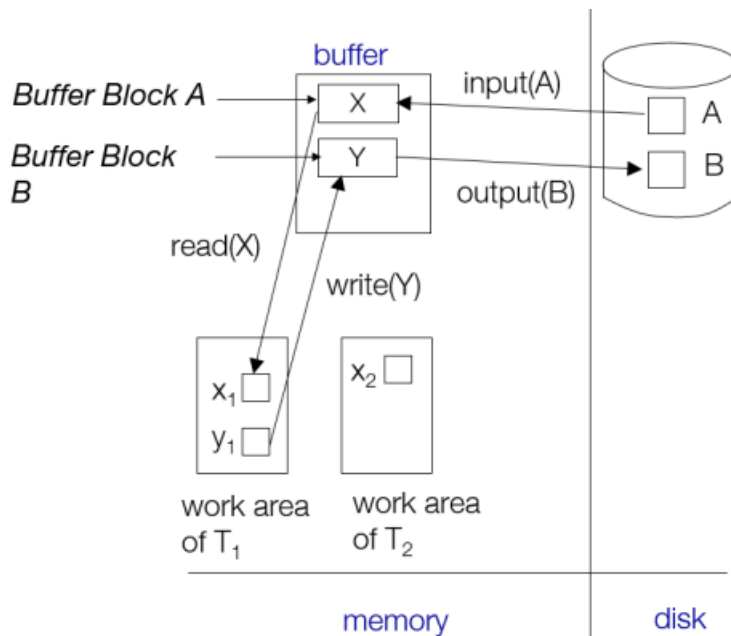
## Concurrency Control and Recovery

- With concurrent transactions, all transactions share a single disk buffer and a single log
  - A buffer block can have data items updated by one or more transactions
- We assume that if **a transaction  $T_i$  has modified an item, no other transaction can modify the same item until  $T_i$  has committed or aborted**
  - That is, the updates of uncommitted transactions should not be visible to other transactions
    - Otherwise how do we perform undo if  $T_1$  updates A, then  $T_2$  updates A and commits, and finally  $T_1$  has to abort?
  - Can be ensured by obtaining exclusive locks on updated items and holding the locks till end of transaction (strict two-phase locking)
- Log records of different transactions may be interspersed in the log

## Example of Data Access with Serial Transaction



## Example of Data Access with Concurrent Transactions



## Recovery Algorithm

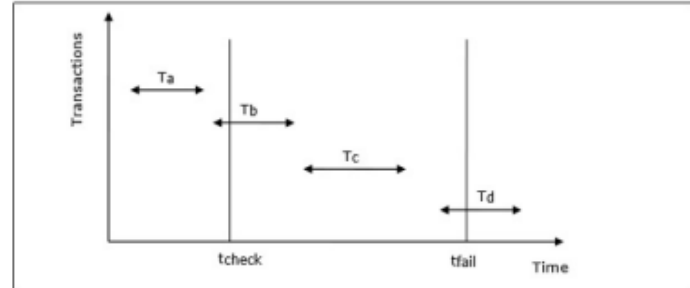
- Logging (during normal operation):
  - **<  $T_i$  start >** at transaction start
  - **<  $T_i$  ,  $X_j$  ,  $V1$  ,  $V2$  >** for each update, and
  - **<  $T_i$  commit >** at transaction end

## Recovery Algorithm (2)

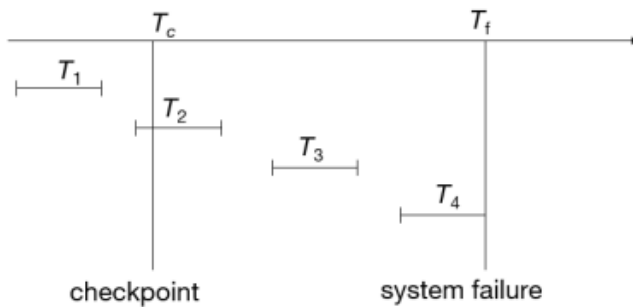
- **Transaction rollback (during normal operation)**
  - Let  $T_i$  be the transaction to be rolled back
  - Scan log backwards from the end, and for each log record of  $T_i$  of the form **<  $T_i$  ,  $X_j$  ,  $V1$  ,  $V2$  >**
    - perform the undo by writing  $V1$  to  $X_j$  ,
    - write a log record **<  $T_i$  ,  $X_j$  ,  $V1$  >** . . . such log records are called **Compensation Log Records (CLR)**
  - Once the record **<  $T_i$  start >** is found stop the scan and write the log record **<  $T_i$  abort >**

### Recovery Algorithm (3): Checkpoints Recap

- Let the time of checkpointing is  $t_{check}$  and the time of system crash is  $t_{fail}$
- Let there be four transactions  $T_a$ ,  $T_b$ ,  $T_c$  and  $T_d$  such that:
  - $T_a$  commits before checkpoint
  - $T_b$  starts before checkpoint and commits before system crash
  - $T_c$  starts after checkpoint and commits before system crash
  - $T_d$  starts after checkpoint and was active at the time of system crash
- The actions that are taken by the recovery manager are:
  - Nothing is done with  $T_a$
  - Transaction redo is performed for  $T_b$  and  $T_c$
  - Transaction undo is performed for  $T_d$



### Recovery Algorithm (4): Checkpoints Recap



- Any transactions that committed before the last checkpoint should be ignored
  - $T_1$  can be ignored (updates already output to disk due to checkpoint)
- Any transactions that committed since the last checkpoint need to be redone
  - $T_2$  and  $T_3$  redone
- Any transaction that was running at the time of failure needs to be undone and restarted
  - $T_4$  undone

### Recovery Algorithm (5): Redo-Undo Phases

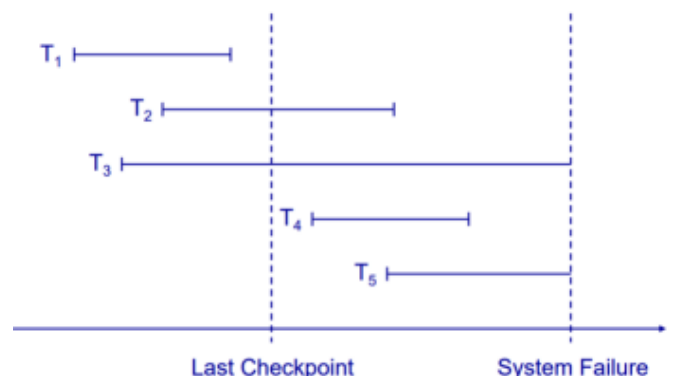
- Recovery from failure:** Two phases
  - Redo phase:** Replay updates of all transactions, whether they committed, aborted, or are incomplete
  - Undo phase:** Undo all incomplete transactions

#### Requirement:

- Transactions of type  $T_1$  need no recovery
- Transactions of type  $T_2$  or  $T_4$  need to be redone
- Transactions of type  $T_3$  or  $T_5$  need to be undone and restarted

#### Strategy:

- Ignore  $T_1$
- Redo  $T_2$ ,  $T_3$ ,  $T_4$  and  $T_5$
- Undo  $T_3$  and  $T_5$



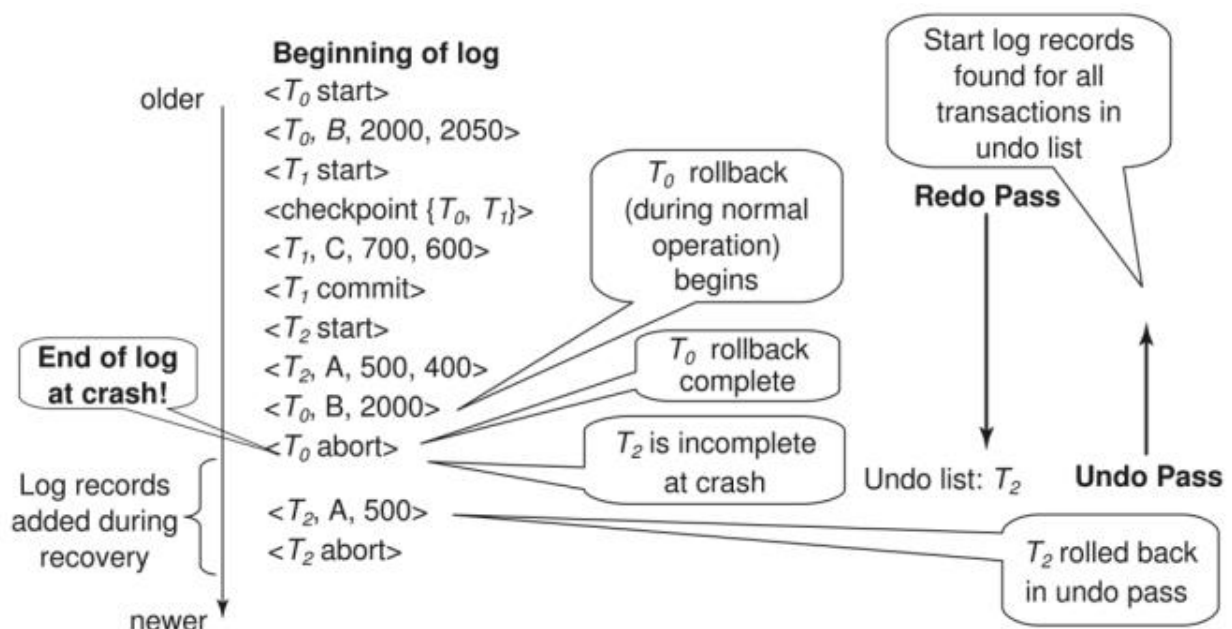
## Recovery Algorithm (6): Redo Phase

- Find last < **checkpoint** L > record, and set undo-list to L
- Scan forward from above < **checkpoint** L > record
  - Whenever a record <  $T_i$  ,  $X_j$  ,  $V_1$  ,  $V_2$  > is found, redo it by writing  $V_2$  to  $X_j$
  - Whenever a log record <  $T_i$  **start** > is found, add  $T_i$  to undo-list
  - Whenever a log record <  $T_i$  **commit** > or <  $T_i$  **abort** > is found, remove  $T_i$  from undo-list
- Steps for the REDO operation are:
  - If the transaction has done INSERT, the recovery manager generates an insert from the log
  - If the transaction has done DELETE, the recovery manager generates a delete from the log
  - If the transaction has done UPDATE, the recovery manager generates an update from the log.

## Recovery Algorithm (7): Undo Phase

- Scan log backwards from end
  - Whenever a log record <  $T_i$  ,  $X_j$  ,  $V_1$  ,  $V_2$  > is found where  $T_i$  is in undo-list perform same actions as for transaction rollback:
    - Perform undo by writing  $V_1$  to  $X_j$
    - Write a log record <  $T_i$  ,  $X_j$  ,  $V_1$  >
  - Whenever a log record <  $T_i$  **start** > is found where  $T_i$  is in undo-list
    - Write a log record <  $T_i$  **abort** >
    - Remove  $T_i$  from undo-list
  - Stop when undo-list is empty That is, <  $T_i$  start > has been found for every transaction in undo-list
- Steps for the UNDO operation are:
  - If the faulty transaction has done INSERT, the recovery manager deletes the data item(s) inserted
  - If the faulty transaction has done DELETE, the recovery manager inserts the deleted data item(s) from the log
  - If the faulty transaction has done UPDATE, the recovery manager eliminates the value by writing the before-update value from the log
- After undo phase completes, normal transaction processing can commence

## Recovery Algorithm (8): Example



## Recovery with Early Lock Release

- Any **index used in processing a transaction**, such as a B+-tree, can be treated as normal data
- To increase concurrency, the B+-tree concurrency control algorithm often allow locks to be released early, in a non-two-phase manner
- As a result of early lock release, it is possible that
  - a value in a B+-tree node is updated by one transaction T1, which inserts an entry (V1, R1), and subsequently
  - by another transaction T2, which inserts an entry (V2, R2) in the same node, moving the entry (V1, R1) even before T1 completes execution
- At this point, we cannot undo transaction T1 by replacing the contents of the node with the old value prior to T1 performing its insert, since that would also undo the insert performed by T2; transaction T2 may still commit (or may have already committed)
- Hence, the only way to undo the effect of insertion of (V1, R1) is to execute a corresponding delete operation
- Support for high-concurrency locking techniques, such as those used for B +-tree concurrency control, which release locks early
  - Supports “logical undo”
- Recovery based on **“repeating history”**, whereby recovery executes exactly the same actions as normal processing
  - including redo of log records of incomplete transactions, followed by subsequent undo
  - Key benefits
    - supports logical undo
    - easier to understand/show correctness
- Early lock release is important not only for indices, but also for operations on other system data structures that are accessed and updated very frequently like:
  - data structures that track the blocks containing records of a relation
  - the free space in a block
  - the free blocks

## Logical Undo Logging

- Operations like B +-tree insertions and deletions release locks early
  - They cannot be undone by restoring old values (physical undo), since once a lock is released, other transactions may have updated the B +-tree
  - Instead, insertions (deletions) are undone by executing a deletion (insertion) operation (known as logical undo)
- For such operations, undo log records should contain the undo operation to be executed
  - Such logging is called logical undo logging, in contrast to physical undo logging
    - Operations are called logical operations
  - Other examples:
    - delete of tuple, to undo insert of tuple
      - allows early lock release on space allocation information
    - subtract amount deposited, to undo deposit
      - allows early lock release on bank balance

## Physical Redo

- Redo information is logged **physically** (that is, new value for each write) even for operations with logical undo
  - Logical redo is very complicated since database state on disk may not be “operation consistent” when recovery starts
  - Physical redo logging does not conflict with early lock release

## Operation Logging: Process

- When operation starts, log  $\langle T_i, O_j, \text{operation-begin} \rangle$ . Here  $O_j$  is a unique identifier of the operation instance
- While the system is executing the operation, it creates update log records in the normal fashion for all updates performed by the operation
  - the usual old-value (**physical undo information**) and new-value (**physical redo information**) is written out as usual for each update performed by the operation;
  - the old-value information is required in case the transaction needs to be rolled back before the operation completes
- When operation completes,  $\langle T_i, O_j, \text{operation-end}, U \rangle$  is logged, where  $U$  contains information needed to perform a logical undo information
  - For example, if the operation inserted an entry in a B+-tree, the undo information  $U$  would indicate that a deletion operation is to be performed, and would identify the B+-tree and what entry to delete from the tree. This is called **logical logging**
  - In contrast, logging of old-value and new-value information is called **physical logging**, and the corresponding log records are called **physical log records**

## Operation Logging (2): Example

- Insert of (key, record-id) pair (K5, RID7) into index I9

$\langle T1, O1, \text{operation-begin} \rangle$   
....  
 $\langle T1, X, 10, K5 \rangle$   
 $\langle T1, Y, 45, \text{RID7} \rangle$   
 $\langle T1, O1, \text{operation-end}, (\text{delete I9, K5, RID7}) \rangle$

} Physical redo of steps in insert

## Operation Logging (3)

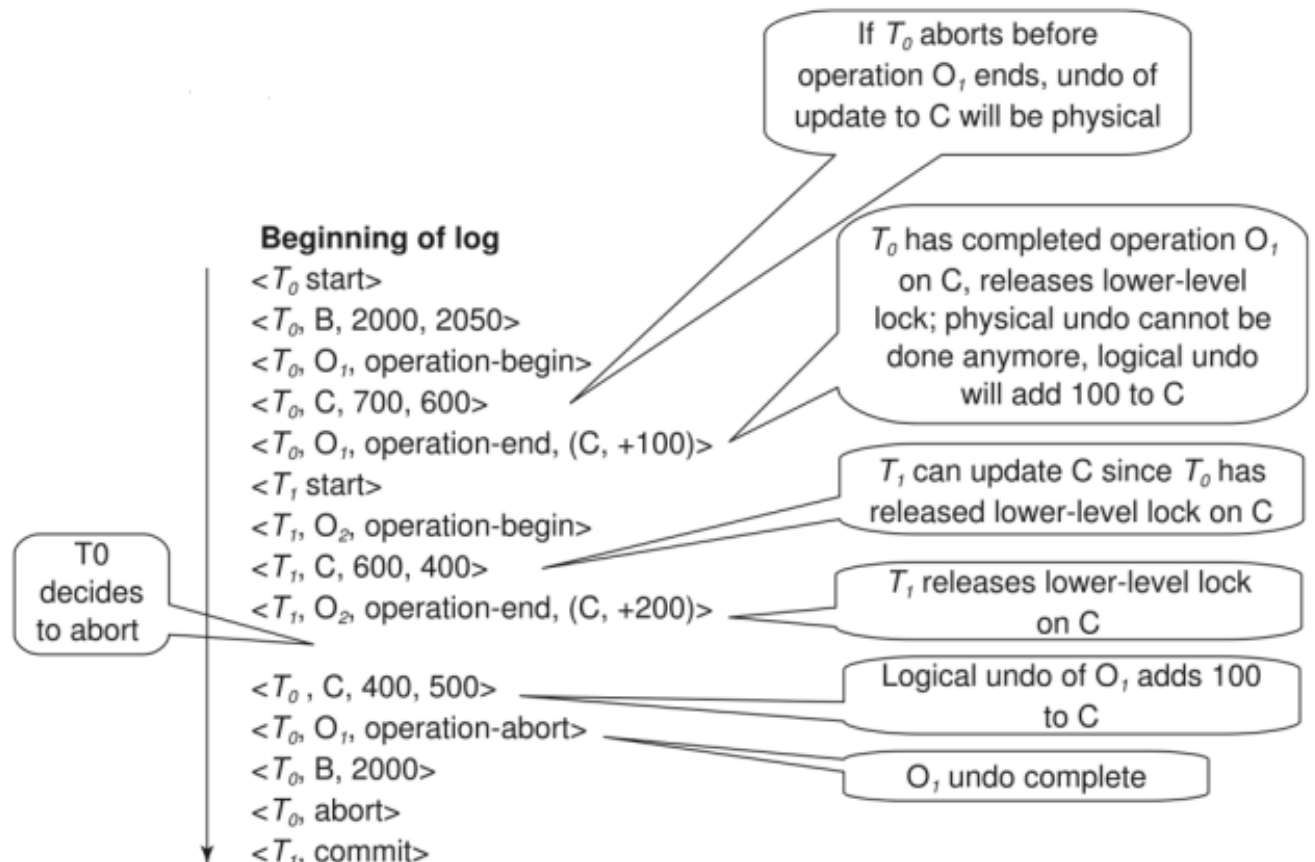
- If crash/rollback occurs before operation completes:
  - the **operation-end** log record is not found, and
  - the physical undo information is used to undo operation
- If crash/rollback occurs after the operation completes:
  - the **operation-end** log record is found, and in this case
  - logical undo is performed using  $U$ ; the physical undo information for the operation is ignored
- **Redo of operation (after crash) still uses physical redo information**

## Transaction Rollback with Logical Undo

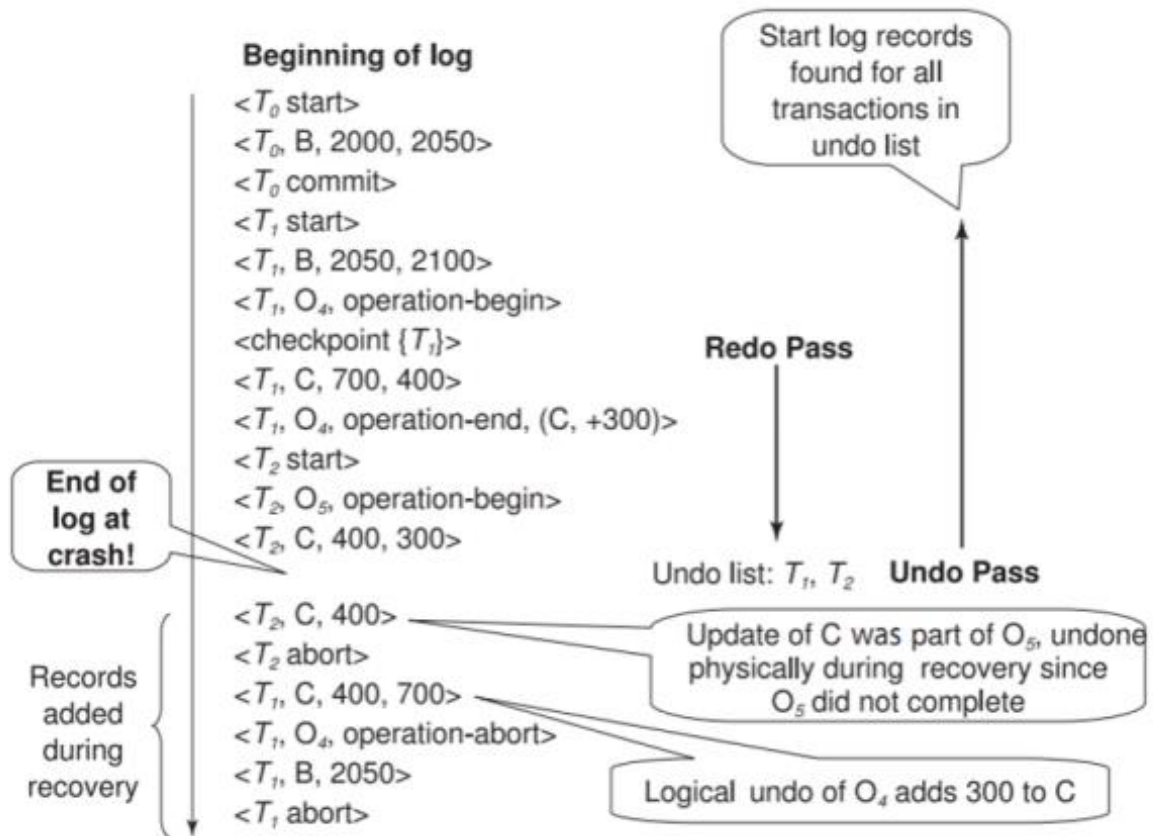
Rollback of transaction  $T_i$ , scan the log backwards

- a) If a log record  $\langle T_i, X, V1, V2 \rangle$  is found, perform the undo and log  $\langle T_i, X, V1 \rangle$
- b) If a  $\langle T_i, O_j, \text{operation-end}, U \rangle$  record is found
  - o Rollback the operation logically using the undo information  $U$ 
    - Updates performed during roll back are logged just like during normal operation execution
    - At the end of the operation rollback, instead of logging an operation-end record, generate a record  $\langle T_i, O_j, \text{operation-abort} \rangle$
  - o Skip all preceding log records for  $T_i$  until the record  $\langle T_i, O_j, \text{operation-begin} \rangle$  is found
- c) If a redo-only record is found ignore it
- d) If a  $\langle T_i, O_j, \text{operation-abort} \rangle$  record is found: skip all preceding log records for  $T_i$  until the record  $\langle T_i, O_j, \text{operation-begin} \rangle$  is found
- e) Stop the scan when the record  $\langle T_i, \text{start} \rangle$  is found
- f) Add a  $\langle T_i, \text{abort} \rangle$  record to the log Note:
  - Cases c) and d) above can occur only if the database crashes while a transaction is being rolled back
  - Skipping of log records as in case d) is important to prevent multiple rollback of the same operation

## Transaction Rollback with Logical Undo



## Failure Recovery with Logical Undo



## Transaction Rollback: Another Example

- Example with a complete and an incomplete operation

```

< T1,start >
< T1, O1, operation-begin>
...
< T1, X, 10,K5 >
< T1, Y , 45, RID7 >
< T1, O1, operation-end, (delete I 9,K5, RID7) >
< T1, O2, operation-begin>
< T1, Z, 45, 70 >
    ← T1 Rollback begins here
< T1, Z, 45 > ← Redo-only log record during physical undo (of incomplete O2)
< T1, Y , . . . , . . . > ← Normal redo records for logical undo of O1
...
< T1, O1, operation-abort> ← What if crash occurred immediately after this?
< T1, abort >
  
```

## Recovery Algorithm with Logical Undo

Basically same as earlier algorithm, except for changes described earlier for transaction rollback

- (**Redo phase**): Scan log forward from last < **checkpoint L** > record till end of log
  - **Repeat history** by physically redoing all updates of all transactions,
  - Create an undo-list during the scan as follows
    - undo-list is set to L initially
    - Whenever < T<sub>i</sub> **start**> is found T<sub>i</sub> is added to undo-list
    - Whenever < T<sub>i</sub> **commit**> or < T<sub>i</sub> **abort**> is found, T<sub>i</sub> is deleted from undo-list

- This brings database to state as of crash, with committed as well as uncommitted transactions having been redone
- Now undo-list contains transactions that are **incomplete**, that is, have neither committed nor been fully rolled back
- (**Undo phase**): Scan log backwards, performing undo on log records of transactions found in undo-list
  - Log records of transactions being rolled back are processed as described earlier, as they are found
    - Single shared scan for all transactions being undone
  - When < Ti **start**> is found for a transaction Ti in undo-list, write a < Ti **abort**> log record.
  - Stop scan when < Ti **start**> records have been found for all Ti in undo-list
- This undoes the effects of incomplete transactions (those with neither **commit** nor **abort** log records). Recovery is now complete.

## Plan for Backup and Recovery

Deciding factors for having a Backup & Recovery setup.

- **Data Importance**
  - How important is the information in your database for your company? For business-critical data you will create a plan that involves making extra copies of your database over the same period and ensuring that the copies can be easily restored when required
- **Frequency of Change**
  - How often does your database get updated? For instance, if critical data is modified daily then you should make a daily backup schedule.
- **Speed**
  - How much time do you need to back up or recover your files? Recovery speed is an important factor that determines the maximum possible time period that could be spent on database backup and recovery.
- **Equipment**
  - Do you have necessary equipment to make backups? To perform timely backups and recoveries, you need to have proper software and hardware resources.
- **Employees**
  - Who will be responsible for implementing your database backup and recovery plan? Ideally, one person should be appointed for controlling and supervising the plan, and several IT specialists (e.g. system administrators) should be responsible for performing the actual backup and recovery of data.
- **Storing**
  - Where do you plan to store database duplicates? In case of Online/Offsite storage you can recover your systems in case of a natural disaster. Storing backups on-site is essential to quick restore. But onsite storage has capacity bottlenecks and high maintenance costs.

## RAID: Redundant Array of Independent Disks

- Disk organization techniques that manage a large numbers of disks, providing a view of a single disk of
  - **high capacity** and **high speed** by using multiple disks in parallel,
  - **high reliability** by storing data redundantly, so that data can be recovered even if a disk fails